

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1403

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Duration :60 Mins
On Class
Total Marks:50

Annexure A

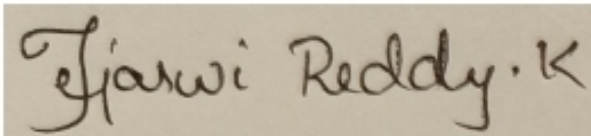
ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I Tejaswi Reddy K(192371036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Q1. Given,
 $w = x * y - z + 20$

i) Lexical Analysis -

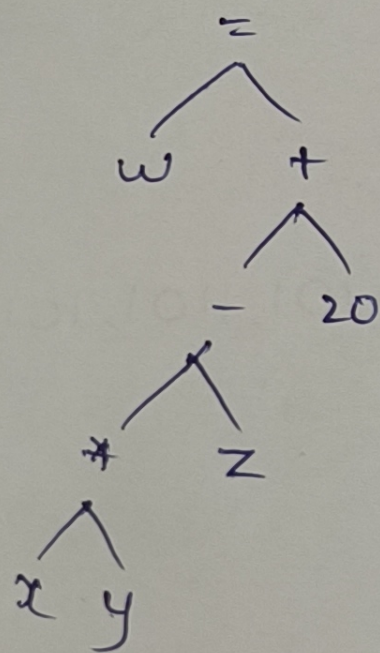
w	Identifier
=	operator
x	Identifier
*	operator
y	Identifier
-	operator
z	Identifier
+	operator
20	Integer.

ii) Syntax Analysis -

Grammar rules -

$$\begin{aligned}
 S &\rightarrow id = E \\
 E &\rightarrow E + T \\
 E &\rightarrow E - T \\
 E &\rightarrow T \\
 T &\rightarrow T * F \\
 T &\rightarrow F \\
 F &\rightarrow id
 \end{aligned}$$

Parse Tree -



iii) Semantic Analysis -

Type checking - w, x, y, z - real (identifiers)

20 - Integer

$x * y$ - real

$y - z$ - real

$z + 20$ - real + integer - Int. Converted to Real

Result $\rightarrow$ Real.

iv) Intermediate Code generation -

$$t_1 = x * y$$

$$t_2 = t_1 - z$$

$$t_3 = \text{int to Real}(20)$$

$$t_4 = t_2 + t_3$$

$$w = t_4$$

v) Code optimization -

$$t_1 = x * y$$

$$t_2 = t_1 - z$$

$$w = t_1 + 20.0$$

vi) Target Code generation - Assembly Code -

LOAD R1, x

MUL R1, y

SUB R1, z

ADD R1, #20.0

STORE w, R1

Q2. i) $R = 1(0+1)^*1$

ii) $R = (0+1)^*01(0+1)^*$

iii) $R = 1^*01^*$

iv) $R = (0+1)^*1(0+1)^*1(0+1)^*$

v) $R = (0+1)^*1$

Q3. i) Given,

$$(E + aa^*)^* = (a^*)^* = a^*$$

$$[\because E + aa^* = a^*] [\because (a^*)^* = a^*]$$

$$\text{Language } L = \{ \epsilon, a, aa, aaa, aaaa, \dots \}$$

It generates strings with only a's.

$$ii) (a^* + b^*)^* abb = (a + b)^* abb$$

The language contains all strings that end with substring abb.

$$L = \{abb, aabb, ababb, aababb, baababb, \dots\}$$

$$iii) (a^* b^*)^* aba$$

$$(a^* b^*)^* = (a + b)^*$$

$$\therefore (a + b)^* aba$$

The language generates strings that end with aba.

$$L = \{aba, aaba, baaba, abababa, abababa, \dots\}$$

$$iv) (a + ab)^*$$

The language generates strings where everything is preceded by a.

$$L = \{aab, aaba, ab, abb, \dots\}$$

$$v) (aba)^* + (a^* b^*)^* = (aba)^* + (a + b)^*$$

The language generates all possible strings.

$$L = \{ab, aba, a, abaab, ababaaba, \dots\}$$

Q4. i) Lex Program with keywords, identifiers, constants,

Operators & delimiters. -

% {

#include <stdio.h>

% }

% %

if { printf ("Keyword: if\n"); }

else { printf ("Keyword: else\n"); }

while { { printf ("Keyword: while\n"); }

[A-Z a-z]([A-Z a-z] | [0-9])^{*} { printf ("Identifier: %s\n", yytext); }

[0-9]⁺ { printf ("Constant: %s\n", yytext); }


```

" + " | " - " | " * " | " / " { printf("Operators: %.s\n", yytext); }
" ( " | " ) " | " { " | " } " | " , " { printf("Delimiter: %.s\n", yytext); }

[ \t \n ] + ;

%. % .

int main()
{
    printf("Enter the Text: \n");
    yylex();
}

int yywrap() {
    return 1;
}

```

ii) Justification -

Identifier - $[A-Z a-z] ([A-Z a-z] | [0-9])^*$

$[A-Z a-z]$ - starts with letter.

$[A-Z a-z] ([0-9])^*$ - Remaining characters can be letters or digits.

Valid - a, sum, a₁, Total Marks.

Invalid - labc, -bc

Keywords - if, else, while.

They are placed before identifiers to avoid conflicts with Identifier.

Valid - if, else, while. (Keywords)

Invalid - if1, else2, while20 (Identifier)

5. i) $(0^* 1^*)^* = (0+1)^*$

Language of $(0^* 1^*)^*$ = $\{\epsilon, 0, 1, 01, 101, 00, 11, \dots\}$

Language of $(0+1)^*$ = $\{\epsilon, 0, 1, 01, 101, 00, 11, \dots\}$

$\therefore$ Both of the expressions generate same language, the expressions are equal.

$$ii) (0+1)^* (0+1)^* = (0+1)^*$$

$$\text{w.k.t, } x^* \cdot x^* = x^*$$

$$x^* = (0+1)^*$$

$$\therefore (0+1)^* (0+1)^* = (0+1)^*$$

$$iii) (0^*)^* = 0^*$$

$$\text{w.k.t, } (x^*)^* = x^*$$

$$\therefore (0^*)^* = 0^*$$

$(0^*)^*$ - generates any no. of zeroes.

0^* - generates any no. of zeroes.

$$iv) (0+1)^* 0 (0+1)^* + (0+1)^* 1 (0+1)^* = (0+1)^*$$

$$= (0+1)^* (0+1)^* (0+1)$$

$$= (0+1)^* (0+1)$$

$$[\because x^* \cdot x^* = x^*]$$

$$= (0+1)^*$$

$$[\because x \cdot x^* = x^*]$$

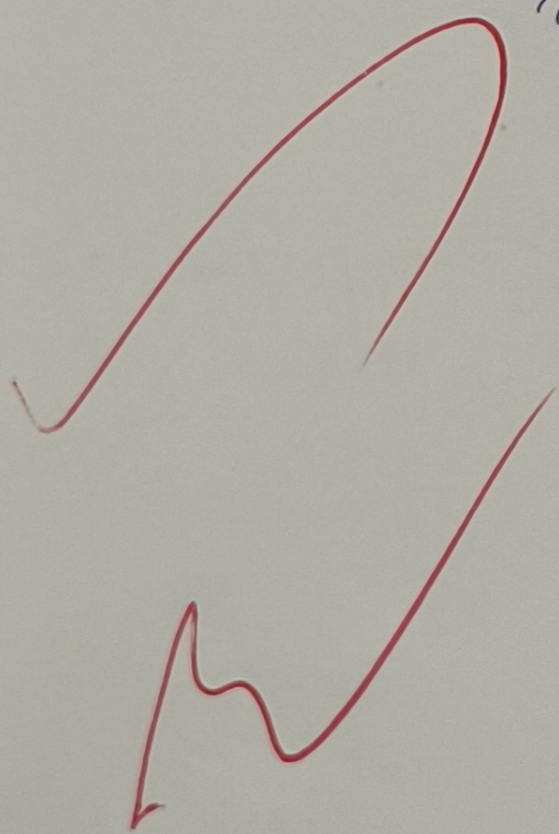
$$v) \epsilon + 0(0)^* = 0^*$$

$$= \epsilon + 00^*$$

$$[\because \epsilon + x x^* = x^*]$$

$$= 0^*$$

//



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation